

Dynamic Workflows com AI Agents e Sub-Agents: Padrões, Frameworks e Aplicações em Produção

OpenClaw Agency Julho de 2026

Resumo

Sistemas multi-agente evoluíram de experimentos de laboratório para produção empresarial em escala. O Gartner relata um aumento de 1.445% em consultas sobre arquiteturas multi-agente entre o primeiro trimestre de 2024 e o segundo trimestre de 2025. Este artigo apresenta os padrões fundamentais de orquestração de agentes de IA — Orchestrator-Worker, Sequential Pipeline, Parallel Fan-Out/Fan-In, Dynamic Handoff e Agent-as-Tool — analisando seus trade-offs, casos de uso e implementações nos principais frameworks do mercado. São discutidos LangGraph, CrewAI, OpenAI Agents SDK, Claude Agent SDK, Microsoft Agent Framework e Spring AI Agent Utils, com uma matriz de compatibilidade entre padrões e ferramentas. O artigo conclui com recomendações práticas para adoção em produção, incluindo estratégias de gerenciamento de contexto, otimização de custos, segurança e observabilidade.

Palavras-chave: agentes de IA, workflows dinâmicos, orquestração multi-agente, padrões de arquitetura, LangGraph, CrewAI

1. Introdução: Por que Agents e Sub-Agents?

A inteligência artificial generativa atingiu um patamar de maturidade em que modelos individuais resolvem tarefas isoladas com alta competência. No entanto, problemas do mundo real raramente são tarefas isoladas. Um processo empresarial típico envolve pesquisa, análise, redação, revisão, aprovação e publicação — cada etapa demandando habilidades, fontes de dados e critérios de qualidade distintos.

É nesse contexto que emergem os sistemas multi-agente. Em vez de um único modelo tentar resolver um problema complexo em uma única chamada, múltiplos agentes especializados colaboram, cada um responsável por uma parte do processo. Cada agente

pode ter seu próprio modelo de linguagem, seu próprio conjunto de ferramentas, suas próprias instruções e seu próprio contexto. O resultado é um sistema mais modular, auditável e resiliente.

O valor fundamental dos sub-agentes não é o paralelismo — é a compressão de contexto. Em sistemas mono-agente, todo o histórico da conversa e todo o conhecimento de domínio precisam caber em uma única janela de contexto. À medida que a tarefa se alonga, o contexto cresce, a qualidade degrada e os custos aumentam. Sub-agentes bem projetados mantêm o contexto do agente pai enxuto, reduzindo o consumo de tokens em mais de 90% e prevenindo a degradação por limites de contexto, conforme reportado por estudos da Epsilla.

A pergunta que organizações enfrentam em 2026 não é mais *se* adotar arquiteturas multi-agente, mas *qual padrão de orquestração* e *qual framework* melhor se adequam a cada caso de uso.

2. O que São Dynamic Workflows?

Dynamic Workflows são arquiteturas de software nas quais múltiplos agentes de IA colaboram de forma coordenada para executar tarefas complexas. Diferentemente de pipelines fixos e monolíticos, workflows dinâmicos permitem que a topologia de execução — quais agentes são chamados, em que ordem, com que frequência — seja definida em tempo de execução com base no contexto da requisição.

O conceito se apoia em três pilares:

- 1. Decomposição:** uma tarefa complexa é dividida em subtarefas menores e mais específicas.
- 2. Delegação:** cada subtarefa é atribuída ao agente mais qualificado para executá-la.
- 3. Coordenação:** os resultados parciais são agregados, refinados e transformados no produto final.

A orquestração pode ser estática (a topologia é definida em tempo de design) ou dinâmica (a topologia emerge durante a execução). A maioria dos sistemas de produção adota uma abordagem híbrida: uma estrutura geral é predefinida, mas decisões de roteamento, replanejamento e especialização são tomadas dinamicamente pelos próprios agentes.

O ecossistema atual converge para cinco padrões fundamentais de orquestração, cada um com características, vantagens e limitações específicas, detalhados nas seções seguintes.

3. Orchestrator-Worker (Supervisor)

Descrição

O padrão Orchestrator-Worker, também conhecido como Supervisor, é o mais difundido em sistemas multi-agente de produção. Um agente orquestrador central recebe a requisição do usuário, decompõe o problema em subtarefas, delega cada subtarefa a um agente especialista (worker), monitora o progresso e sintetiza o resultado final.

Quando Utilizar

- Workflows complexos onde a decomposição da tarefa requer julgamento contextual
- Planos de execução que variam conforme a entrada do usuário
- Cenários onde agentes especialistas precisam de diferentes modelos, ferramentas ou bases de conhecimento
- Situações que exigem replanejamento dinâmico com base em resultados intermediários

Trade-offs

O padrão adiciona 15% a 30% de latência e 20% a 40% de overhead de tokens devido às múltiplas chamadas de ida e volta entre orquestrador e workers. Em contrapartida, oferece a mais alta flexibilidade e auditabilidade entre os padrões de orquestração.

Exemplo Real

O sistema de orquestração do Claude Agent SDK (Anthropic) utiliza este padrão como primitiva fundamental. Um agente principal (orchestrator) utiliza a ferramenta `agent.asTool()` para delegar sub-tarefas a agentes especializados, mantendo controle sobre o fluxo geral enquanto cada sub-agente opera com contexto e ferramentas isolados. Este modelo é análogo ao hub-and-spoke descrito pela Anthropic em sua documentação do SDK.

A Microsoft Agent Framework implementa o mesmo padrão através de workflow graphs com transições explícitas, substituindo o modelo puramente conversacional do AutoGen por uma abordagem baseada em grafos de estado.

4. Sequential Pipeline (Prompt Chaining)

Descrição

No padrão Sequential Pipeline, agentes são encadeados em ordem linear predefinida. Cada agente processa a saída do anterior, aplicando uma transformação específica. O fluxo é determinístico: a sequência de etapas é conhecida antes da execução.

Quando Utilizar

- Processos com dependências lineares claras entre etapas
- Refinamento progressivo (rascunho → revisão → polimento)
- Pipelines de transformação de dados com estágios bem definidos
- Cenários onde cada etapa requer um modelo ou ferramenta diferente

Quando Evitar

- Tarefas que podem ser executadas em paralelo
- Workflows que exigem backtracking ou iteração entre etapas
- Roteamento dinâmico baseado em resultados intermediários
- Problemas onde a próxima etapa depende de uma decisão contextual

Exemplo Real

Um escritório de advocacia implementou um pipeline de quatro agentes para geração de contratos: (1) seleção de template baseado no tipo de contrato, (2) customização de cláusulas conforme dados do cliente, (3) verificação de compliance regulatório e (4) avaliação de risco jurídico. Cada agente opera com um modelo especializado e acesso a bases de dados distintas — jurisprudência, legislação, precedentes — sem que nenhum agente individual precise ter acesso a todas elas.

Implementação

LangGraph oferece suporte nativo a pipelines sequenciais através de grafos acíclicos dirigidos (DAGs), onde cada nó representa um agente e as arestas definem a ordem de execução. O checkpointing integrado permite pausar e retomar o pipeline em qualquer ponto, facilitando a auditoria e a correção de erros.

5. Parallel Fan-Out / Fan-In

Descrição

No padrão Parallel Fan-Out/Fan-In, múltiplos agentes processam a mesma tarefa simultaneamente, cada um de sua perspectiva ou especialidade. Os resultados são agregados em uma etapa de Fan-In, que pode usar votação, média ponderada, sumarização via LLM ou qualquer outra estratégia de consenso.

Quando Utilizar

- Análise multi-perspectiva (técnica, financeira, criativa, regulatória)
- Ensemble reasoning e quorum voting para decisões críticas
- Cenários onde a latência é prioridade e o processamento paralelo reduz o tempo total
- Tarefas que se beneficiam de múltiplos pontos de vista independentes

Estratégias de Agregação

- 1. Votação Majoritária:** cada agente produz uma classificação ou escolha; a opção com mais votos vence.
- 2. Média Ponderada:** agentes contribuem com pontuações; pesos refletem a confiança ou especialização de cada um.
- 3. Sumarização por LLM:** um agente agregador recebe todas as saídas e sintetiza um resultado coerente.
- 4. Debate Estruturado:** agentes argumentam suas posições em rodadas antes da votação final.

Exemplo Real

Uma instituição financeira implementou análise de ações com quatro agentes paralelos: (1) análise fundamentalista (balanços, indicadores financeiros), (2) análise técnica (padrões de gráfico, tendências), (3) análise de sentimento (notícias, redes sociais) e (4) análise ESG (sustentabilidade, governança). Cada agente opera de forma independente com fontes de dados próprias, e um meta-agente consolida os resultados em um relatório de investimento com recomendação final.

Considerações de Custo

Padrões concorrentes podem causar picos significativos de consumo de tokens. É recomendável designar modelos menores e mais econômicos para agentes de análise menos críticos, reservando modelos maiores apenas para a etapa de agregação.

6. Dynamic Handoff (Routing)

Descrição

No padrão Dynamic Handoff, agentes decidem dinamicamente quando transferir o controle para outro agente mais especializado. Diferentemente do Orchestrator-Worker, onde o orquestrador mantém o controle durante todo o processo, no Handoff apenas um agente opera por vez, e a transferência de controle é definitiva ou semi-definitiva.

Quando Utilizar

- O especialista certo para a tarefa não é conhecido de antemão
- Requisitos de especialização emergem durante o processamento
- Sistemas de suporte ao cliente com múltiplos domínios de conhecimento
- Cenários onde cada interação exige um conjunto diferente de ferramentas

Tipos de Handoff

O OpenAI Agents SDK distingue dois tipos fundamentais:

- 1. Ownership Transfer:** o agente atual transfere completamente o controle para outro agente, que assume a responsabilidade pela interação. O agente original não recupera o controle a menos que o novo agente decida transferir de volta.
- 2. Agent-as-Tool:** o agente atual invoca outro agente como uma ferramenta, mantendo o controle e aguardando o resultado. Este é o modelo do Orchestrator-Worker visto anteriormente.

Exemplo Real

Um CRM de telecomunicações utiliza um agente de triagem inicial que identifica a natureza da solicitação do cliente e encaminha dinamicamente para o agente especializado apropriado: agente técnico (problemas de conexão, configuração de equipamento), agente financeiro (faturas, planos, cobranças) ou agente de contas (mudança de titularidade, cancelamento). O agente de triagem não precisa conhecer todas as respostas — apenas saber para quem delegar cada pergunta.

Suporte nos Frameworks

O OpenAI Agents SDK trata handoffs como cidadãos de primeira classe com a função `handoff()`, incluindo validação de entrada e saída. O Claude Agent SDK implementa handoff através de sub-agents programáticos, onde cada sub-agent opera em contexto isolado. O Microsoft Agent Framework oferece handoff como transição explícita em workflow graphs.

7. Agent-as-Tool

Descrição

Agent-as-Tool é um padrão híbrido onde um agente é exposto como uma ferramenta que outros agentes podem invocar. Diferentemente de ferramentas tradicionais (APIs, funções, bancos de dados), um Agent-as-Tool possui autonomia para planejar, executar múltiplos passos e tomar decisões dentro de seu escopo antes de retornar o resultado.

Quando Utilizar

- Uma subtarefa é complexa demais para uma única chamada de função
- A subtarefa requer seu próprio ciclo de planejamento e execução
- Diferentes partes do sistema precisam da mesma capacidade especializada
- Isolamento de contexto e ferramentas sensíveis é necessário

Diferença entre Handoff e Agent-as-Tool

A distinção é sutil mas importante. No Handoff (ownership transfer), o agente que recebe a tarefa assume o controle da interação com o usuário. No Agent-as-Tool, o agente chamador mantém o controle e trata o agente chamado como um recurso computacional. A escolha entre os dois padrões depende de quem deve "dirigir" a interação: o agente chamador (Agent-as-Tool) ou o agente chamado (Handoff).

Exemplo Real

Em um sistema de geração de relatórios empresariais, um agente coordenador utiliza três Agent-as-Tool especializados: (1) um agente de consulta a bancos de dados SQL que planeja e executa consultas complexas, (2) um agente de visualização que gera gráficos e tabelas a partir de dados brutos e (3) um agente de redação que produz narrativa em linguagem natural. O coordenador invoca cada um como ferramenta, recebe os resultados e monta o relatório final.

Suporte nos Frameworks

O Claude Agent SDK oferece suporte nativo através de `agent.asTool()`, permitindo que qualquer agente seja utilizado como ferramenta por outro agente. O OpenAI Agents SDK segue abordagem similar. O Spring AI Agent Utils implementa o padrão via Task tool, onde cada sub-agent é executado em contexto próprio e isolado.

8. Ferramentas e Frameworks

O ecossistema de frameworks para orquestração multi-agente amadureceu significativamente entre 2024 e 2026. A seguir, uma análise dos principais frameworks disponíveis.

8.1 LangGraph (LangChain)

LangGraph é o framework mais maduro e adotado para workflows complexos de agentes, com mais de 90 mil estrelas no GitHub e 47 milhões de downloads no PyPI. Utiliza um modelo de grafo de estado dirigido com arestas condicionais, permitindo controle fino sobre o fluxo de execução.

Diferenciais: checkpointing com time-travel debugging, estado tipado, observabilidade via LangSmith. Adotado em produção por Uber, LinkedIn, Klarna, Replit e Elastic.

Melhor para: workflows complexos que exigem auditabilidade e controle granular.

8.2 CrewAI

CrewAI oferece um modelo de times baseados em papéis (role-based crews) com 20 mil estrelas no GitHub. Seu diferencial é a velocidade de setup: modo hierárquico com manager agent automático, reportado como 5,76 vezes mais rápido que LangGraph em tarefas de QA.

Melhor para: prototipagem rápida, provas de conceito e pipelines de conteúdo. Sessenta por cento das empresas Fortune 500 estão em fase de exploração com CrewAI.

Limitação: falta de controle fino de fluxo e state management primitivo para cenários complexos.

8.3 OpenAI Agents SDK

O SDK de agentes da OpenAI trata o conceito de agente como primitiva fundamental e handoffs como cidadãos de primeira classe. Oferece `handoff()` function, `agent.asTool()` para padrão manager-style, guardrails declarativos para input e output, e tracing built-in.

Estado: versão 0.12.x, substitui a Assistants API (depreciação programada para agosto de 2026).

Melhor para: organizações já consolidadas no ecossistema OpenAI que precisam de multi-agent com baixa latência.

8.4 Claude Agent SDK (Anthropic)

O SDK da Anthropic oferece agent loop com sub-agents, suporte a MCP (Model Context Protocol), hooks e permissions. Utiliza o mesmo harness do Claude Code, com sub-agents em contexto isolado e paralelização nativa.

Padrões suportados: sub-agents programáticos, agentes baseados em arquivos (`.claude/agents/`), dynamic workflows via scripts JavaScript e agent teams.

Melhor para: equipes que já utilizam Claude Code, workflows de desenvolvimento e agentes conscientes do codebase.

8.5 Microsoft Agent Framework

Sucessor empresarial do AutoGen (que entrou em manutenção), o Microsoft Agent Framework atingiu GA no primeiro trimestre de 2026. Utiliza workflow graphs com transições explícitas — diferentemente do modelo conversacional do AutoGen — e oferece suporte built-in a MCP com mais de 50 ferramentas.

Diferenciais: pipeline completo de middleware para logging, PII redaction e auditoria; integração nativa com Azure AI Services.

Melhor para: empresas no ecossistema Azure com requisitos rigorosos de compliance.

8.6 Spring AI Agent Utils

Framework para o ecossistema Spring/Java, oferece sub-agentes hierárquicos com Task tool inspirado no Claude Code. Suporta multi-model routing (diferentes LLMs por sub-agent) e protocolo A2A. Inclui quatro sub-agentes built-in: Explore, General-Purpose, Plan e Bash.

Melhor para: organizações que já utilizam Spring Boot e ecossistema Java.

8.7 Matriz de Compatibilidade

Framework	Orchestrator	Sequential	Parallel	Handoff	Group Chat	A2A	MCP
LangGraph	Nativo	Nativo	Nativo	Via custom	Via custom	-	Via LangChain
CrewAI	Hierarchical	Sequential	Nativo	Limitado	-	Sim	Community
OpenAI Agents SDK	Agent-as-Tool	Manual	Via handoffs	Nativo	-	-	Sim
Claude Agent SDK	Agent tool	Programático	Sub-agents	Sub-agents	Agent Teams	Planejado	Nativo
MS Agent Framework	Workflow graphs	Workflow graphs	Concurrent	Handoff	GroupChat legado	Sim	Built-in
Spring AI	Task tool	Programático	Task tool	Task tool	-	Sim	Sim

9. Considerações de Produção

9.1 Gerenciamento de Contexto

Contextos expandem rapidamente em sistemas multi-agente. Cada interação entre agentes adiciona tokens ao histórico. Técnicas de compactação incluem sumarização seletiva e pruning de mensagens entre agentes. A regra prática recomendada é: se o contexto acumulado excede 70% do limite do modelo, aplique compactação.

9.2 Otimização de Custos

Múltiplos agentes significam múltiplas invocações de modelo. A estratégia recomendada é designar modelos diferentes para cada agente conforme a complexidade da tarefa — modelos menores e mais econômicos para tarefas rotineiras, modelos maiores apenas para raciocínio profundo e síntese final.

9.3 Segurança

Cada agente deve operar com o princípio do menor privilégio: acesso apenas às ferramentas e dados necessários para sua função. Guardrails devem ser implementados em múltiplos pontos — antes da entrada, durante chamadas de ferramentas e antes da saída. Circuit breakers previnem que falhas em cascata se propaguem pelo sistema.

9.4 Human-in-the-Loop

Vários padrões suportam participação humana: observadores em group chat, revisores em ciclos maker-checker e escalação em handoff. Pontos de verificação obrigatórios tornam a orquestração síncrona, e o estado deve ser persistido nos checkpoints para permitir retomada sem replay.

9.5 Observabilidade

O stack mínimo de observabilidade para sistemas multi-agente combina LangSmith (tracing de agentes), OpenTelemetry (métricas de infraestrutura) e dashboards de workflow engine. Toda operação e handoff deve ser instrumentado, com rastreamento de latência, consumo de recursos e qualidade de saída por agente.

10. Conclusão e Tendências

Os padrões de orquestração multi-agente amadureceram de conceitos acadêmicos para pilares de produção empresarial. A escolha do padrão correto depende do tipo de problema, dos requisitos de latência e custo, e da maturidade técnica da equipe.

Arquitetura Híbrida Recomendada para 2026

A recomendação que emerge da literatura e dos casos de produção é uma arquitetura em camadas:

- 1. Temporal para durabilidade:** retries, state persistence, pausas para human-in-the-loop.
- 2. LangGraph para raciocínio do agente:** expressividade de grafo com melhor tooling de produção.
- 3. Kafka + A2A para coordenação inter-serviço:** barramento de eventos para comunicação entre agentes em diferentes domínios.

4. **Observabilidade desde o primeiro dia:** LangSmith, OpenTelemetry e dashboards dedicados.
5. **Evolução incremental:** comece com topologia estática e adicione dinamismo gradualmente.

Tendências Emergentes

Seis tendências moldam o futuro dos dynamic workflows:

1. **DAGs (Directed Acyclic Graphs):** workflows modelados como grafos, com LangGraph como expoente máximo.
2. **Orquestração Event-Driven:** agentes reagem a eventos em barramentos como Kafka e RabbitMQ.
3. **Modelo Actor:** cada agente como ator com caixa postal, popularizado por AutoGen e MAF.
4. **Roteamento Dinâmico por Dificuldade:** classificadores estimam a complexidade da query e alocam recursos proporcionais.
5. **Orquestração Federada:** coordenação entre cloud e edge através de fronteiras de confiança, com protocolo A2A como bloco fundamental.
6. **Workflows Auto-Otimizáveis:** meta-agentes ajustam a topologia do workflow com base em feedback de execuções anteriores.

Princípio Fundamental

Comece com um agente generalista, especialize apenas quando houver evidência empírica de que a especialização melhora resultados. Os gatilhos legítimos para especialização incluem requisitos divergentes de modelo (visão versus classificação), limites de segurança (dados sensíveis versus públicos), compliance regulatório (finanças, saúde) e evidência mensurável de que um agente especialista supera o generalista na tarefa específica.

11. Bibliografia

1. Vellum AI. The 2026 Guide to AI Agent Workflows, 2025. Disponível em: <https://www.vellum.ai/blog/agentic-workflows-emerging-architectures-and-design-patterns>
2. Tzolov, C. Spring AI Agentic Patterns (Part 4): Subagent Orchestration. Spring Blog, 2026. Disponível em: <https://spring.io/blog/2026/01/27/spring-ai-agentic-patterns-4-task-subagents/>
3. Ricki (Epsilla). The 3 Essential Sub-Agent Patterns for Production-Grade AI Systems. Epsilla Blog, 2026. Disponível em:

<https://www.epsilla.com/blogs/2026-03-14-ai-sub-agent-patterns>

4. Microsoft. AI Agent Orchestration Patterns. Azure Architecture Center, 2026. Disponível em: <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/ai-agent-design-patterns>

5. Sempf, A.; Hooker, A. Agentic AI patterns and workflows on AWS. AWS Prescriptive Guidance, 2025. Disponível em: <https://docs.aws.amazon.com/prescriptive-guidance/latest/agentic-ai-patterns/introduction.html>

6. Zylos Research. Agent Workflow Orchestration Patterns: DAG, Event-Driven, and Actor Models, 2026. Disponível em: <https://zylos.ai/research/2026-04-14-agent-workflow-orchestration-patterns>

7. Pucek, B. Orchestration Patterns for AI Agent Workflows. The Thinking Company, 2026. Disponível em: <https://thinking.inc/en/blue-ocean/agentic/agent-orchestration-patterns>

8. Devarajan, V. Common Workflow Patterns for AI Agents. GUVI Blog, 2026. Disponível em: <https://www.guvi.in/blog/common-workflow-patterns-for-ai-agents>

9. Digital Applied. Multi-Agent Orchestration: 5 Patterns That Work in 2026, 2026. Disponível em: <https://www.digitalapplied.com/blog/multi-agent-orchestration-5-patterns-that-work>

10. Lushbinary. Multi-Agent AI Orchestration Patterns: Supervisor, Swarm, Pipeline & Router, 2026. Disponível em: <https://lushbinary.com/blog/multi-agent-orchestration-patterns-supervisor-swarm-pipeline-router-guide>

11. Shakudo. Enterprise Agentic AI Workflow Patterns for 2025, 2025. Disponível em: https://cdn.prod.website-files.com/625447c67b621ab49bb7e3e5/69388ca4cdb5836ee83b10f5_69388ca257d8a9675e92aeb8_agentic-ai-workflow-patterns-whitepaper.pdf

12. Shakudo. Multi-Agent Systems Transform Enterprise AI in 2026. Disponível em: https://cdn.prod.website-files.com/625447c67b621ab49bb7e3e5/696fdd9dcee4a9c14251480b_696fdd9c3876035a277d2a6f_multi-agent-systems-2026-trends-whitepaper.pdf

13. WitnessAI. AI Agent Orchestration: Managing Multi-Agent Workflows, 2026. Disponível em: <https://witness.ai/blog/ai-agent-orchestration>

14. OpenAI. Agents SDK: Orchestration and handoffs, 2026. Disponível em: <https://developers.openai.com/api/docs/guides/agents/orchestration>

15. OpenAI. Agents SDK: Handoffs, 2026. Disponível em: <https://openai.github.io/openai-agents-python/handoffs/>

16. APIScout. OpenAI Agents SDK: Architecture Patterns 2026. Disponível em: <https://apiscout.dev/guides/openai-agents-sdk-architecture-patterns-2026>

17. Anthropic. Claude Agent SDK: Subagents in the SDK, 2026. Disponível em: <https://code.claude.com/docs/en/agent-sdk/subagents>
18. Anthropic. Claude Code: Orchestrate subagents at scale with dynamic workflows, 2026. Disponível em: <https://code.claude.com/docs/en/workflows>
19. Anthropic. Claude Code: Agent Teams, 2026. Disponível em: <https://code.claude.com/docs/en/agent-teams>
20. SubAgent.Dev. Orchestration Patterns, 2026. Disponível em: <https://subagent.developersdigest.tech/patterns>
21. Gheware, R. LangGraph vs CrewAI vs AutoGen: Complete AI Agent Framework Comparison 2026, 2026. Disponível em: <https://devops.gheware.com/blog/posts/langgraph-vs-crewai-vs-autogen-comparison-2026.html>
22. Grokipedia. AI Agent Orchestration, 2026. Disponível em: https://grokipedia.com/page/AI_Agent_Orchestration
23. Stackvivo. Agentic AI & Multi-Agent Systems: Advanced Guide, 2026. Disponível em: <https://stackvivo.ai/blog/agentic-ai-multi-agent-systems-guide>
24. RockB. Microsoft Agent Framework 2026: AutoGen Successor Explained, 2026. Disponível em: <https://baeseokjae.github.io/posts/microsoft-agent-framework-2026/>
25. Microsoft Learn. Multi-agent patterns, 2026. Disponível em: <https://learn.microsoft.com/en-us/agents/architecture/multi-agent-patterns>
26. JetThoughts. LangGraph vs CrewAI vs AutoGen: Open Source Alternatives 2025, 2026. Disponível em: <https://jetthoughts.com/blog/autogen-crewai-langgraph-ai-agent-frameworks-2025>
27. Agent Market Cap. LangGraph vs AutoGen vs CrewAI vs DSPy: The 2026 Multi-Agent Framework Decision Guide, 2026. Disponível em: <https://agentmarketcap.ai/blog/2026/04/11/langgraph-autogen-crewai-dspy-multi-agent-orchestration-2026>
28. OpenAgents. CrewAI vs LangGraph vs AutoGen vs OpenAgents: Best AI Agent Framework, 2026. Disponível em: <https://openagents.org/blog/posts/2026-02-23-open-source-ai-agent-frameworks-compared>
29. Meta Intelligence. LangGraph vs CrewAI vs AutoGen: AI Agent Framework Comparison [2026], 2025. Disponível em: <https://www.meta-intelligence.tech/en/insight-ai-agent-frameworks>
30. WinBuzzer. Anthropic Shows How to Scale Claude Code with Subagents and MCP, 2026. Disponível em: <https://winbuzzer.com/2026/03/24/anthropic-claude-code-subagent-mcp-advanced-patterns-xcxwbn>